

Spring Boot internals

How Spring Boot actually works under the hood: startup, the IoC container, auto-configuration, dependency injection, the bean lifecycle, configuration, profiles, starters, embedded servers, and Actuator. The tone is how I would talk each one through with a teammate; the investigation and deployment questions are fully spoken.

1. What is Spring Boot, why was it introduced, and what does it solve?

So Spring Boot is a layer **on top of the Spring Framework** that makes Spring apps quick to start and easy to run. Plain Spring was powerful but painful: mountains of XML and manual configuration, and you had to wire and deploy everything yourself. Spring Boot solves that with **auto-configuration** (sensible defaults based on what is on the classpath), **starter dependencies** (curated, version-compatible bundles), and an **embedded server** (so the app is a runnable jar, not a war you deploy). The theme is **convention over configuration**: it makes the common case just work while letting you override anything. And no, it is not a replacement for Spring, it **is** Spring plus the boilerplate removed.

2. Spring Framework v/s Spring Boot.

	Spring Framework	Spring Boot
Configuration	Manual (XML or Java config)	Auto-configured with defaults
Dependencies	Pick and version each yourself	Starters bundle compatible versions
Server	Deploy a war to an external server	Embedded server, runnable jar
Setup effort	High	Low

The one-liner: Spring gives you the **container and features**; Spring Boot gives you a **fast, opinionated way to assemble and run** a Spring app.

3. What are the advantages and disadvantages?

Advantages: fast setup, auto-configuration, starters, embedded server, production-ready features (Actuator), and it is the default for microservices. **Disadvantages:** the **auto-configuration can feel like magic** (hard to know why a bean exists until you learn to read the report), the **fat jar is large**, and if you fight the conventions it can be more work than plain Spring. So the trade is speed and defaults for a bit of "what is happening under the hood" opacity.

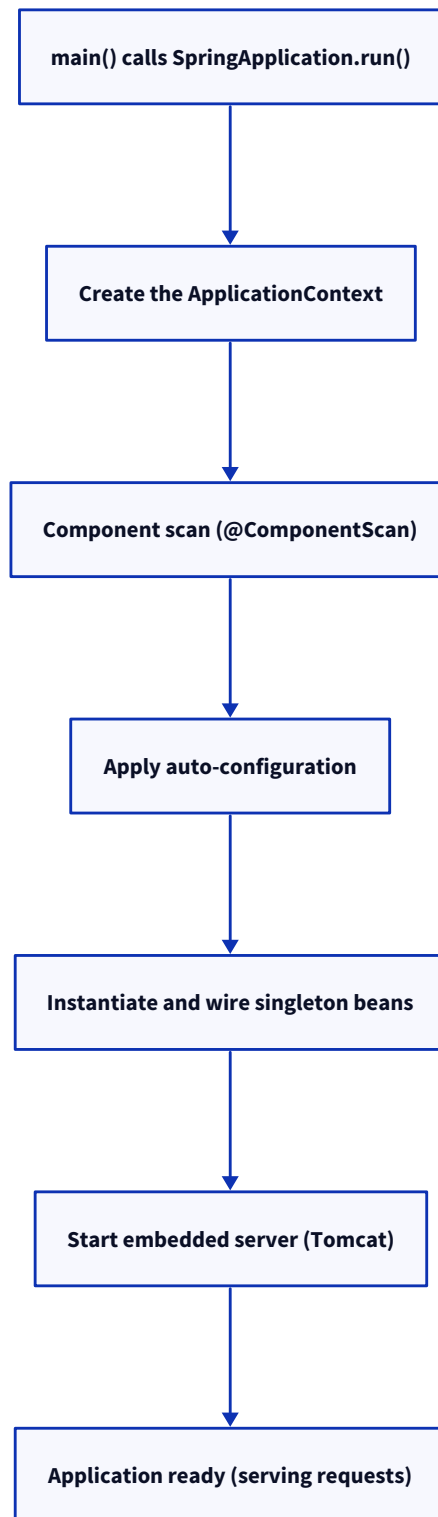
4. Why is Spring Boot so popular for microservices, and what are its key features?

Because a microservice wants to be **small, self-contained, and quick to spin up**, and Spring Boot fits that exactly: each service is a **standalone runnable jar with its own embedded server**, no shared app server, easy to containerise, with **Actuator** giving health and metrics out of the box for orchestration. The **key features** are the same list: auto-configuration, starters, embedded server, Actuator, externalised configuration, and profiles. Put together, you get a service you can build, ship in a container, and monitor with almost no ceremony.

5. Explain the architecture and what happens when a Spring Boot app starts.

The entry point is a `main()` that calls `SpringApplication.run()`, and that one call does a lot.

`SpringApplication` **creates the `ApplicationContext`** (the IoC container), **scans for components**, applies **auto-configuration**, **instantiates and wires the singleton beans** (running their lifecycle callbacks), **starts the embedded server**, and finally publishes an **application-ready event**. So `main()` is just the trigger; `SpringApplication.run()` is where the container is built and the app comes alive.



Follow-ups. What does `main()` actually do? Almost nothing itself, it hands off to `SpringApplication.run(App.class, args)`. When are `@PostConstruct` hooks run? During bean initialisation, before the app is marked ready.

6. IoC container, ApplicationContext, and BeanFactory.

The **IoC (Inversion of Control) container** is the heart of Spring: instead of your code creating its dependencies, the **container creates and wires them for you** and hands them over, that inversion is the whole idea. There are two container interfaces. **BeanFactory** is the basic one (lazy, minimal). **ApplicationContext** extends it and adds what you actually want: **eager singleton initialisation**, event publishing, internationalisation, and AOP integration.

	BeanFactory	ApplicationContext
Level	Basic container	Full-featured (extends BeanFactory)
Bean creation	Lazy	Eager for singletons
Extras	None	Events, i18n, AOP, resource loading

In practice you always use **ApplicationContext**, **BeanFactory** is the underlying abstraction.

7. What is @SpringBootApplication, and which annotations does it combine?

@SpringBootApplication is a **convenience annotation on your main class that bundles three**:

- **@Configuration** (via **@SpringBootConfiguration**): marks the class as a source of bean definitions.
- **@EnableAutoConfiguration**: turns on auto-configuration.
- **@ComponentScan**: scans for your components.

You **can** replace it with those three written out separately, it is purely shorthand. Knowing what it expands to matters because it explains the next two questions (scanning and auto-config).

8. How does component scanning work, and where should the main class live?

@ComponentScan tells Spring to **find your @Component / @Service / @Repository / @Controller classes and register them as beans**. The key detail: by default it scans the **package of the main class and all sub-packages**. So the base package is determined by **where your @SpringBootApplication class sits**, which is why the main class should live in a **root/top-level package** above everything else. Put it too deep and half your components silently do not get scanned, a classic "why is my bean null" bug.

Follow-ups. *Bean not being picked up?* Check it is in a package under the main class. *Scan a package outside the main class's tree?* Add it explicitly with `@ComponentScan(basePackages = ...)`.

9. How does auto-configuration work, and how do you exclude one?

Auto-configuration **configures beans automatically based on what is on the classpath and what you have not already defined**. **@EnableAutoConfiguration** makes Spring read a list of auto-configuration classes (from `META-INF/spring/...AutoConfiguration.imports`), each guarded by **@Conditional** annotations: **@ConditionalOnClass** (this library is present), **@ConditionalOnMissingBean** (you have not defined your own), and so on. So if H2 is on the classpath and you defined no `DataSource`, it makes one; define your own and it **backs off**. To **exclude** an auto-configuration you use `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)` or the `spring.autoconfigure.exclude` property.

Follow-ups. *How do you see what auto-configured?* Run with `--debug` for the auto-configuration report (what matched and why). *How does it back off for your bean?* `@ConditionalOnMissingBean`.

10. What is dependency injection, why does it matter, and how does Spring resolve it?

DI means the **container supplies a class's dependencies from outside** rather than the class creating them, which is the IoC idea in practice. It matters because it **decouples** classes from concrete implementations (depend on an interface, swap the impl, mock it in tests). Spring **resolves** a dependency by **type** first (find a bean of the required type), and if there is ambiguity it falls back to the bean **name** or the `@Qualifier` / `@Primary` hints (next question).

11. Constructor v/s setter v/s field injection, and can Spring inject final fields?

Three styles, and the guidance is firm:

Style	Use
Constructor	Recommended. Required deps, allows <code>final</code> (immutable), easy to test, fails fast
Setter	Optional or reconfigurable dependencies
Field	Convenient but discouraged, cannot be <code>final</code> , hidden deps, needs reflection to test

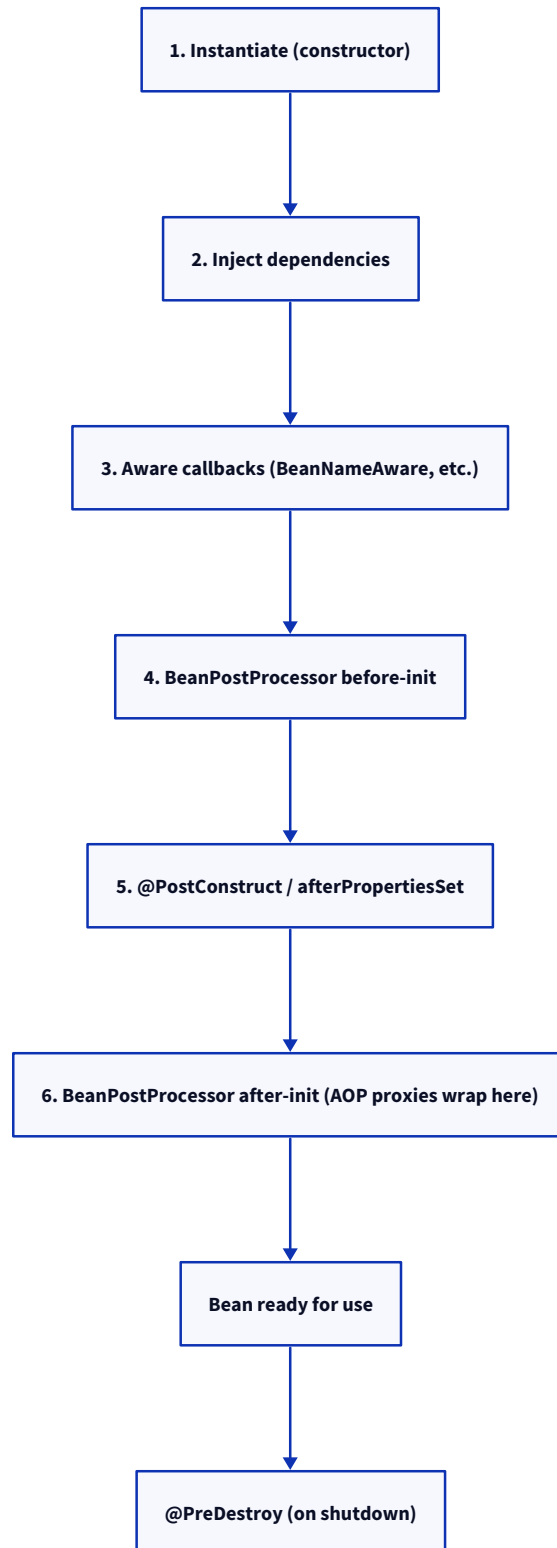
Constructor injection is preferred because the dependencies are explicit, the fields can be `final` (guaranteed set, immutable), you can construct the class with mocks in a test without Spring, and a missing dependency (or a circular one) fails at startup. And yes, Spring **can inject into final fields, but only via constructor injection** (the final field is assigned in the constructor); field injection cannot be final.

12. What happens if multiple beans of the same type exist (@Qualifier and @Primary)?

By default Spring cannot decide and throws a `NoUniqueBeanDefinitionException`. You disambiguate two ways: `@Primary` marks one bean as the default choice when several match, and `@Qualifier("name")` picks a specific one at the injection point. Use `@Primary` for the usual default and `@Qualifier` when a particular consumer needs a specific implementation.

13. What is a Spring bean, and what is its lifecycle?

A **bean is just an object the Spring container creates, wires, and manages**. Its lifecycle is container-driven: **instantiate** the bean, **inject dependencies**, run **aware** callbacks, then **BeanPostProcessor before-init**, then **init** (`@PostConstruct`, then `InitializingBean`), then **BeanPostProcessor after-init** (this is where AOP proxies like `@Transactional` wrap the bean), at which point it is **ready**. On shutdown it runs **destroy** (`@PreDestroy`). The hooks you actually use are `@PostConstruct` (set up once dependencies are wired) and `@PreDestroy` (clean up).



14. What are the bean scopes?

Scope decides how many instances the container makes and how long they live:

Scope	Lifetime
singleton	Default , one instance per container
prototype	A new instance every time it is requested
request	One per HTTP request (web)
session	One per HTTP session (web)
application / websocket	One per servlet context / WebSocket session

You can also define **custom scopes**. The one that trips people: **singleton is per Spring container**, not per JVM, and it is shared across all threads.

15. Lazy v/s eager initialisation, and @Lazy.

By default the container **eagerly creates singleton beans at startup**, which surfaces wiring problems immediately (fail fast) but makes startup slower. `@Lazy` defers a bean's creation **until it is first used**, which speeds startup and avoids building things you might not need, at the cost of moving any failure to first use. So eager is the safe default; reach for `@Lazy` for genuinely expensive or rarely-used beans.

16. How does Spring manage singleton beans, and are they thread-safe?

Spring keeps **one instance per container** and hands the same object to everyone who injects it. And here is the critical point: **a singleton bean is NOT automatically thread-safe**. Because one instance is shared across all request threads, if it holds **mutable instance state**, you have a race. The rule is to keep singleton beans **stateless** (or make any shared state thread-safe). Most service and repository beans are naturally stateless, which is why this usually just works, until someone adds a mutable field.

Follow-ups. *So how do I hold per-request state?* Local variables (on the stack, per thread), or a request-scoped bean, not a field on the singleton.

17. application.properties v/s application.yml.

Both configure the app; the difference is format. `application.properties` is **flat key-value** (`server.port=8080`); `application.yml` is **hierarchical YAML** (nested, less repetition), which reads better for grouped config but is whitespace-sensitive. They are functionally equivalent, pick one per project for consistency.

18. How does Spring Boot load configuration, and what is the precedence order?

Spring Boot pulls configuration from **many sources and layers them**, with more specific sources overriding general ones. Roughly high to low precedence: **command-line arguments** > **environment variables / system properties** > **profile-specific files** (`application-{profile}.properties`) > `application.properties` / `.yml` > **defaults in code**. So you can bake defaults into `application.properties`, override per environment with a profile file, and override again at deploy time with an env var or a command-line flag, without touching the jar. That layering is what "externalised configuration" means.

Follow-ups. *Why does a command-line flag win?* It is the most specific, deploy-time override. *Where do secrets fit?* As env vars or a secrets manager, never committed to the properties file (see Q21).

19. @Value v/s @ConfigurationProperties.

`@Value("${my.prop}")` injects a **single property** into a field, fine for one-off values.

`@ConfigurationProperties` binds a **whole group of related properties to a typed POJO** (`app.mail.*` into a `MailProperties` object). `@ConfigurationProperties` is **preferred for large or grouped configuration** because it is type-safe, validated, and keeps related settings together instead of scattering dozens of `@Value` s. It also supports **relaxed binding**, so `my-prop` , `my.prop` , `myProp` , and `MY_PROP` all bind to the same field, which is why an env var in caps maps cleanly to a property.

Follow-ups. *Validate config?* Put `@Validated` on the `@ConfigurationProperties` class and use bean-validation annotations, so a bad config fails at startup. *Why not @Value everywhere?* It does not scale, no grouping, no type safety.

20. What are Spring profiles, and how do you use them?

Profiles let you have **different configuration and beans per environment** (dev, test, prod). You mark environment-specific beans with `@Profile("dev")` and put environment-specific settings in `application-{profile}.properties` . You **activate** a profile with `spring.profiles.active` (as a property, env var `SPRING_PROFILES_ACTIVE` , or a command-line flag), and you can activate **several at once** (comma-separated). There is a **default** profile when none is set. So the same jar runs in dev or prod purely by switching the active profile, no rebuild.

Follow-ups. *Prod v/s dev config?* Different `application-prod.properties` / `application-dev.properties` , and `@Profile` beans for things like a real v/s an in-memory data source. *Multiple active profiles?* Yes, they layer.

21. Secrets management and configuration best practices.

The main rule: **never commit secrets** (passwords, API keys) to `application.properties` in the repo. Instead inject them as **environment variables** or pull them from a **secrets manager / vault** at runtime. Beyond that, the practices I would hold to: **externalise anything environment-specific** (do not hardcode), use `@ConfigurationProperties` **with validation** for grouped config, use **profiles** for per-environment differences, and keep sensible **defaults** in `application.properties` for local dev. (*Adapt to your own setup.*)

22. What are Spring Boot starters?

A starter is a **curated dependency bundle** for a capability, so instead of hand-picking a dozen libraries and hoping the versions are compatible, you add one starter. `spring-boot-starter-web` pulls in Spring MVC, Jackson, and embedded Tomcat, all at compatible versions; `spring-boot-starter-data-jpa` brings JPA and Hibernate; `spring-boot-starter-test` brings JUnit, Mockito, and friends. So starters remove the **dependency-picking and version-juggling** work.

23. How does Spring Boot manage dependency versions (the BOM)?

Through a **BOM (Bill of Materials)**: `spring-boot-starter-parent` (or the imported `spring-boot-dependencies` BOM) declares a **tested, compatible set of versions** for a huge list of libraries, so when you add a dependency you **omit the version** and inherit the managed one. That is why Spring Boot dependencies "just work together". You **can override** a managed version (set a property like `<spring.version>` or specify an explicit version), but you do it carefully, because you are stepping outside the tested combination.

Follow-ups. *Why omit versions?* The BOM guarantees a compatible set. *Risk of overriding?* You break the tested compatibility matrix, so verify it.

24. Embedded servers: what is supported, and how do they differ?

Spring Boot **embeds the servlet container inside your jar**, so there is no external server to install. The options are **Tomcat** (the default), **Jetty**, and **Undertow**. They are largely interchangeable: **Tomcat** is the well-understood default, **Jetty** is lightweight and common for embedded use, and **Undertow** is high-performance and low-footprint. You swap one for another by excluding the Tomcat starter and adding the Jetty/Undertow one. And yes, you **can run without an embedded server** (a non-web app, or a war deployed to an external container).

Follow-ups. *How does embedded Tomcat work?* Spring Boot starts a Tomcat instance in-process during startup and registers your app with it, so the jar is self-running. *When would you pick Undertow?* When footprint and throughput matter and you have measured a benefit.

25. WAR v/s JAR, and fat v/s thin jar.

Spring Boot's default and preferred packaging is an **executable ("fat" or "uber") jar**: it bundles your code, all dependencies, **and** the embedded server, so `java -jar app.jar` runs the whole thing, ideal for containers and microservices. A **WAR** is for the older model of deploying to an **external** servlet container, you use it only when you must deploy into existing infrastructure. A **thin jar** externalises the dependencies (smaller artifact, deps fetched/mounted separately), which some deployment setups prefer, but the fat jar is the norm.

26. How would you package and deploy a Spring Boot app for production?

The way I would do it: build the **executable fat jar** (`mvn package / gradle bootJar`), then **containerise** it, a Dockerfile on a slim JRE base, or Spring Boot's built-in image building (`buildpacks / bootBuildImage`). I would **externalise all config** so the same image runs in every environment, driven by env vars and the active profile, and **inject secrets** from the platform, not the image. In production I would run it on an orchestrator (Kubernetes), wire the **Actuator health endpoints to the liveness/readiness probes**, set the **JVM heap to the container's memory limit**, and ship logs and metrics out. So: one immutable fat-jar image, config and secrets injected at runtime, health wired to the platform. (*Adapt to your own pipeline.*)

27. What is Spring Boot Actuator, and why does it matter?

Actuator adds **production-ready endpoints** that expose the app's health and internals over HTTP (or JMX), which is exactly what you need to **operate a service**. The common endpoints: `/health` (up/down, wired to k8s probes), `/info` (build/app info), `/metrics` (JVM and app metrics, usually scraped by Prometheus), `/env` (the resolved configuration), and `/beans` (every bean in the context). You can add **custom health indicators** (check a downstream dependency and fold it into `/health`). The important safety point: **secure Actuator**, several endpoints (`/env`, `/beans`, `/heapdump`) leak sensitive detail, so you expose only what you need (`management.endpoints.web.exposure.include`) and put auth in front.

Follow-ups. *Why is `/health` important?* Kubernetes uses it for liveness/readiness, so it drives restarts and traffic. *Custom health indicator?* Implement `HealthIndicator` to report a dependency's status.

28. Your application fails to start because a bean cannot be created. How would you investigate?

The good news is Spring makes this easy, so I start by **reading the failure message**: Boot prints a clear **"APPLICATION FAILED TO START"** block with a **Description** and an **Action** telling you what went wrong and often how to fix it. From there the usual causes: a **missing bean/dependency** (nothing satisfies a required type), **ambiguous beans** (two of the same type, needs `@Qualifier / @Primary`), a **circular dependency** (A

needs B needs A), a **missing config property**, or the **port already in use**. If it is auto-configuration related, I rerun with `--debug` for the auto-configuration report to see what did and did not get configured. So: read Boot's failure block first (it is genuinely helpful), identify which of those cases it is, and fix that. *(Adapt to a real one you have hit.)*

29. If Spring Boot provides auto-configuration, why do developers still write configuration classes?

What they are testing: whether you understand the limits of auto-configuration, that it is a starting point, not the whole story.

Because auto-configuration only knows about **common, generic cases**, it cannot know **your domain**. You still write `@Configuration` classes to **define your own beans** (a domain service, a client for a third-party API), to **customise or override a default** (a `RestTemplate` with your timeouts and interceptors, a custom `ObjectMapper`), to **integrate a library that has no starter**, and to do **conditional wiring** your app needs. So auto-config handles the boilerplate defaults, and your config classes handle everything specific to you. The two work together: auto-config backs off (`@ConditionalOnMissingBean`) precisely so your bean wins.

30. Why does Spring Boot make heavy use of conditional annotations?

What they are testing: whether you grasp that auto-configuration has to be **smart and non-intrusive**, and conditionals are how.

Because auto-configuration runs for **everyone**, so it must only configure a thing **when it actually makes sense**, and never step on what you have defined. `@Conditional` annotations are the mechanism: `@ConditionalOnClass` (only if this library is on the classpath), `@ConditionalOnMissingBean` (only if you have not defined your own), `@ConditionalOnProperty` (only if this flag is set). That is what lets one set of auto-config classes adapt to thousands of different apps and **back off gracefully** the moment you take over. Without conditionals, auto-config would blindly create beans you did not want and clash with yours.

31. How would you design the package structure for a large Spring Boot application with dozens of modules?

What they are testing: architectural judgment at scale, and whether you favour feature cohesion over layer-splitting.

The way I would approach it: **package by feature, not by layer**. The common mistake is top-level `controllers/`, `services/`, `repositories/` packages, which scatter one feature across the codebase and couple everything. Instead each feature (`order`, `payment`, `user`) gets its **own package** holding its controller, service, and repository together, so a feature is self-contained and its internals can stay package-private. I keep the **main class in a root package** above the features so component scanning covers them all, put genuinely shared code in a **common / shared** module, and for real separation at "dozens of modules" scale I would use a **multi-module build** (Maven/Gradle modules) so the boundaries are enforced by the build, not just convention, with dependencies pointing one direction (features depend on shared, never the reverse). So: feature cohesion first, enforced module boundaries as it grows. *(Adapt to your own codebase.)*

32. What engineering guidelines would you establish for Spring Boot development across a large organisation?

What they are testing: whether you can set standards that keep dozens of teams consistent and out of trouble.

The guidelines I would mandate: **constructor injection everywhere** (no field injection); **externalise all config** with profiles, and **never commit secrets** (use a vault/env vars); a **consistent error model** across services (standard exception handling and response shape); **Actuator health wired to the platform probes and secured**; **package by feature** with a shared module; **rely on the BOM** for dependency versions rather than pinning ad hoc; **tests at the right level** (unit + slice tests + Testcontainers for integration); and **observability baked in** (metrics, structured logs, tracing) from the start. The meta-rule is **do not fight the conventions**, Spring Boot is opinionated, and teams that work with its grain move faster than teams that reinvent it. *(Adapt to your own org.)*

33. Looking back, which Spring Boot features had the greatest impact on productivity, and which are most commonly misused?

What they are testing: perspective and honesty, can you weigh the trade-offs of the tools you use daily.

The **biggest productivity wins** are the ones that removed the setup tax: **auto-configuration and starters** (you go from empty project to running app in minutes, no XML, no version juggling) and the **embedded server** (a runnable jar per service, which is what made Spring Boot the default for microservices). **Actuator** is the quiet hero on the ops side. On the **most misused**: **field injection** (convenient, but hides dependencies and hurts testing), **@Value sprawl** where `@ConfigurationProperties` belonged, **treating singleton beans as thread-safe** and adding mutable state, **exposing Actuator endpoints** without securing them, and **leaning on auto-config as magic** without ever learning to read the report, so nobody understands why a bean exists. The pattern is that Spring Boot's convenience is a double-edged sword: it is fastest when you understand what it is doing for you, and most dangerous when you treat it as a black box. *(This is a judgment question, so a defensible take with reasons is what matters.)*

34. How does Spring Boot handle concurrent requests, and is it single- or multi-threaded?

What they are testing: whether you understand the thread-per-request model, which is the root of every Spring Boot concurrency question.

Spring Boot is **multi-threaded**. The embedded server (Tomcat by default) keeps a **pool of worker threads** (around 200 by default), and each incoming request is handled on **its own thread** from that pool, so many requests run **concurrently**. Your controller and service code therefore runs on many threads at once.



This is exactly why bean thread-safety matters: your **singleton beans are shared across all those request threads** (see Q16), so a mutable field on a singleton is a race waiting to happen. The mental model to hold: one shared set of beans, many threads calling into them at the same time.

Follow-ups. *So a slow request blocks others?* No, each is on its own thread, but if all pool threads are busy, new requests queue, hence thread-pool sizing matters. *Where is the pool configured?*

`server.tomcat.threads.max` and friends.

35. What are the common concurrency challenges in Spring Boot applications?

What they are testing: whether you can name the real traps that come from the thread-per-request model.

The recurring ones:

- **Mutable state on singleton beans:** the single most common bug, a shared bean with an instance field that request threads race on (keep beans stateless, see Q16).
- **Thread pool exhaustion:** every worker thread is blocked on a slow downstream call (no timeout), so the pool fills and new requests queue or time out. Fix with timeouts, circuit breakers, and bounded pools.
- **Connection pool sizing:** too few database connections for the number of worker threads means threads wait for a connection; too many overwhelms the database.
- **Shared caches and counters:** non-thread-safe structures (a plain `HashMap` cache, a `count++`) corrupt under concurrent access, use `ConcurrentHashMap`, atomics, or proper synchronisation.
- **@Async and thread pools:** offloaded work needs its own sized executor, and context (security, request scope, trace ids) does not automatically propagate to the async thread.

The theme is the same: anything **shared across request threads** must be stateless or thread-safe.

36. Your API slows down under heavy traffic. How would you investigate thread-related issues?

What they are testing: a methodical approach to a saturated service, and knowledge of the thread-dump technique.

The way I would work it: first confirm from **metrics** whether it is thread-related, is the **worker thread pool saturated** (active threads at max, requests queueing), and are latencies climbing while CPU is not? That pattern says threads are **blocked**, not busy. Then I take a **thread dump** (`jstack` or Actuator's `/threaddump`) and look at what the worker threads are doing: if most are **BLOCKED** or **WAITING on a slow downstream call or a lock**, that is the culprit. The usual root causes are a **missing timeout on a downstream** (so threads pile up waiting) or a **database connection pool** too small for the load (threads waiting for a connection). The fixes follow: add **timeouts** on external calls, a **circuit breaker** so a down dependency fails fast instead of consuming the pool, and right-size the **thread and connection pools**. So: confirm pool saturation from metrics, thread-dump to see what they are stuck on, and fix the blocking cause, not just the pool size. (*Adapt to a real incident you have had.*)

Follow-ups. *Just increase the thread pool?* Rarely the fix, if threads block forever a bigger pool only delays saturation. *Actuator help here?* Yes, `/metrics` for pool usage and `/threaddump` for the stacks.

37. Common mistakes.

- **Field injection instead of constructor injection**, which hides dependencies and cannot be `final` or easily tested.
- **Main class in the wrong (too-deep) package**, so component scanning misses your beans.
- **Hardcoding configuration values** instead of externalising them.
- **Misusing @Value for complex configuration** where `@ConfigurationProperties` fits.
- **Creating multiple beans of the same type unintentionally**, causing ambiguity errors.
- **Ignoring profile-specific configuration**, so dev settings leak into prod.
- **Exposing sensitive Actuator endpoints** (`/env`, `/beans`, `/heapdump`) without auth.
- **Assuming singleton beans are thread-safe**, then adding mutable state to them.
- **Overriding managed dependency versions** without checking compatibility.
- **Ignoring the auto-configuration report** (`--debug`) when debugging why a bean exists or does not.